Recursive Tensor Networks for Efficient Abstract Reasoning

Summary, Motivation & Architecture

Project Proposal

December 2025

Motivation: The "Less is More" Paradigm

Context: Tiny Recursive Models (TRM) show that parameter efficiency > depth for logic.

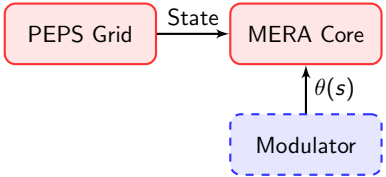
What TRM Changed

- **Parameter Sharing:** Reusing weights forces learning **Universal Rules** vs. memorizing positions.
- **Data Efficiency:** $O(1)$ parameters prevents overfitting on small data.

My Unique Contributions

- **Geometric Bias:** Applying recursion to a **2D Lattice (PEPS)** to respect grid topology.
- **Enforcing Area Law:** Unlike Transformers (Volume Law), We strictly constrain information flow to the boundary, forcing the model to learn causal objects.
- **Modulated Logic:** Introducing **Scale-Dependent Weights** to evolve logic from pixels to objects.

Functional Architecture: Engine vs. Steering



Mandatory (The Engine)

- **PEPS Grid:** Enforces Area Law (Locality).
- **MERA Unit:** Recursive compression.

Adaptive (The Steering)

- **Modulator:** Adapts logic per scale.
- **TTT:** Updates weights at inference.

What is "Reasoning"?

We define reasoning physically as **Renormalization Group (RG) Flow**.

- **The Disentangler (Sorting):**
  - *Action:* Rotates basis to separate **Signal** (Shape) from **Noise** (Texture).
  - *Why Critical:* Prevents mixing irrelevant details during pooling.
- **The Isometry (Abstraction):**
  - *Action:* Maps a group of pixels to a single **Concept Token**.
  - *Why Critical:* Reduces dimensionality while preserving topology.

Core Questions: The f-function

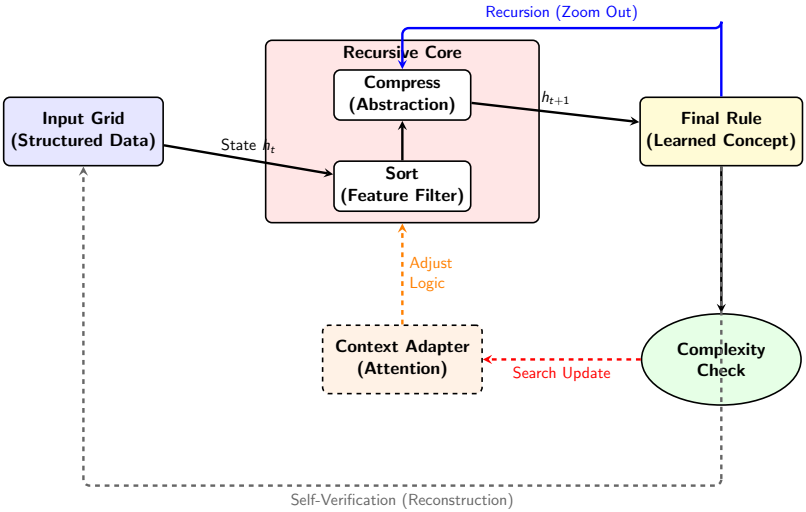
Investigating the transition function $h_{t+1} = f(h_t, \theta)$.

1. **Is f Scale-Invariant?**
   - *Hypothesis:* No. Logic must shift from Edge Detection (UV) to Object Movement (IR). $\rightarrow$ **Requires Modulation**.
2. **Is f Local?**
   - *Hypothesis:* Yes. ARC objects obey the Area Law. Global attention is wasteful. $\rightarrow$ **Requires PEPS**.
3. **Is f Reversible?**
   - *Hypothesis:* Yes. Reasoning should be generative. $\rightarrow$ **Requires Unitary Tensors**.

Broader AI Contributions

- **Abstraction as Renormalization:**
  Formalizing concept formation as the flow from UV (pixels) to IR (rules).
- **Causation vs. Correlation:**
  Replacing All-to-All Attention (Volume Law) with Nearest-Neighbor Constraints (Area Law).
- **Glass-Box Interpretability:**
  Latent states are always valid 2D grids, allowing direct visualization of the "thought process."

System Blueprint



The Challenge: Representation vs. Reality

The Context: The ARC benchmark requires reasoning about 2D grids, objects, and local interactions.

The Mismatch:

- **Standard Deep Learning (Vector Space):** Flattens data into 1D vectors.
- **Consequence:** This destroys spatial locality. A pixel at $(0, 0)$ and $(0, 1)$ might be far apart in the flattened vector.

The Goal: We need a data structure that:

1. Preserves the **2D Geometry** of the input naturally.
2. Handles the exponential size of the state space efficiently.

The "Curse of Dimensionality"

Definition: The explosive growth of the state space as the system size increases.

Mathematical Reality

Consider a grid of N pixels, where each pixel can be one of d colors.

- **Total States:** d^N
- **Example:** For a tiny 10×10 binary grid ($N = 100, d = 2$):

$$2^{100} \approx 10^{30} \text{ states}$$

Implication: We cannot represent the probability distribution or state vector explicitly. We must find a compressed representation that captures the *relevant* structure without storing 10^{30} numbers.

Tensors: A Generalization of Arrays

In this framework, we view data structures through linear algebra.

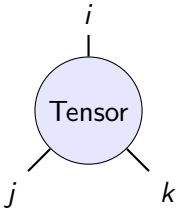
- **Scalar (Rank-0):** A single number. $s \in \mathbb{R}$.
- **Vector (Rank-1):** A 1D array. $v_i \in \mathbb{R}^d$.
- **Matrix (Rank-2):** A 2D array. $M_{ij} \in \mathbb{R}^{d \times d}$.
- **Tensor (Rank-k):** A k-dimensional array. $T_{i_1 i_2 \ldots i_k}$.

CS context: An RGB image is a Rank-3 tensor (Height, Width, Color Channels).

Graphical Notation (Penrose Notation)

To manage complex operations, we use **Graph Theory** instead of equations.

- **Node (Shape):** Represents a Tensor.
- **Edge (Leg):** Represents an index (dimension).
- **Open Edge:** An unsummed index (Output dimension).



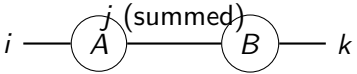
A Rank-3 Tensor T_{ijk}

Contraction = Computation

Definition: Connecting two legs together represents **Summation** over that index (Einstein Summation).

Example: Matrix Multiplication

$$C = A \cdot B \implies C_{ik} = \sum_j A_{ij} B_{jk}$$



ARC Implication: We can represent complex reasoning steps as a network of tensors "contracting" (processing) information locally.

The Power of Factorization (SVD)

Concept: Any large matrix can be decomposed into smaller matrices.

$$M = U \cdot \Sigma \cdot V^{\dagger}$$

Truncated SVD (Compression): If we only keep the top k singular values, we get the best rank-k approximation of the original data.

- **Original:** $N \times N$ parameters.
- **Compressed:** $2Nk$ parameters.
- **Key Insight:** If data has structure (patterns), k can be very small.

Bond Dimension: The Measure of Complexity

In Tensor Networks, that value k is called the **Bond Dimension** (χ).

- χ is the rank of the edge connecting two tensors.
- It represents the **bandwidth of information flow**.

Conceptual Example:

- $\chi = 1$**:** No correlation. Pixels are independent (White Noise).
- **Low** χ**:** Simple correlations (Patterns, Shapes).
- **High** χ**:** Complex/Random correlations (Encryption, Chaos).

For ARC: We constrain χ to force the model to find simple, explainable patterns.

Matrix Product States (MPS)

Definition: Breaking a high-dimensional vector into a 1D chain of local tensors.

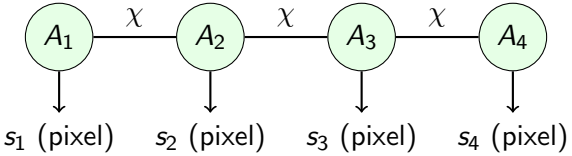
$$\Psi(s_1, s_2, \ldots, s_N) \approx \mathrm{Tr}(A^{[1]s_1} A^{[2]s_2} \ldots A^{[N]s_N})$$

- Instead of one giant tensor, we have N small tensors.
- Each tensor has 3 legs:
  1. **Physical (s):** The data (pixel color).
  2. **Left Virtual (α):** Input context from left neighbor.
  3. **Right Virtual (β):** Output context to right neighbor.

CS Parallel: MPS $\approx$ Hidden Markov Model

Analogy:

- **HMM/RNN:** Current state depends on input + previous hidden state.
- **MPS:** Tensor at site i depends on physical index s_i + virtual bond from $i-1$.



Difference: MPS is a quantum linear algebra generalization, handling superpositions, but structurally it is a "chain of dependencies."

Why 1D is insufficient for ARC

The Snake Problem: If we use a 1D structure (like an MPS or standard Transformer) on a 2D grid, we must "snake" the path.

(0,0) is next to (0,1), but far from (1,0).

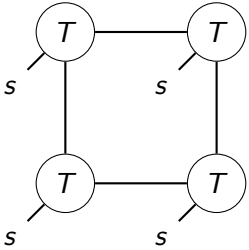
Loss of Locality:

- Vertical neighbors become distant in the chain.
- The model wastes capacity learning "jump" correlations that are actually just local vertical neighbors.

Projected Entangled Pair States (PEPS)

Definition: A Tensor Network that natively matches the 2D lattice geometry.

- Every tensor $T^{[x,y]}$ has **4 Virtual Legs** (North, South, East, West).
- **Exact Mapping:** The tensor graph is isomorphic to the pixel grid.



The "Area Law" (Sparsity Constraint)

This is the most crucial concept for data efficiency.

Volume Law vs. Area Law

- **Volume Law (Random/Dense):** Info content scales with total pixels (N^2). (Most Neural Nets).
- **Area Law (Structured):** Info content scales with the **boundary size** (N). (PEPS).

Implication: PEPS naturally models objects where the interior is uniform, and complexity exists only at the edges. This describes 99% of ARC tasks (solid shapes, filled regions).

13. PEPS as a Logical Substrate

We treat the PEPS grid not just as compression, but as a simulation board.

- **Quasiparticles:** An "object" moving in ARC corresponds to an "excitation" moving across the PEPS bonds.
- **Local Updates:** We can update the logic at grid point (x, y) based solely on its 4 neighbors.

Benefit: We restrict the search space to **Locally Causal Rules**.

Coarse-Graining (Renormalization)

Goal: To reason abstractly, we must move from Pixels $\rightarrow$ Objects.
Mechanism: Coarse-Graining (similar to Pooling in CNNs).

- We group 2×2 blocks of tensors into single "Super-Tensors."
- We repeat this recursively.

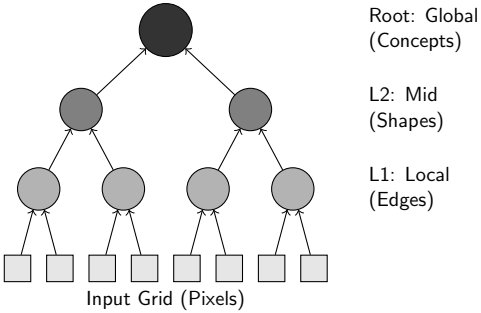
The Problem with Standard Pooling: Standard Max-Pool/Avg-Pool is **destructive**. It throws away texture information, making it impossible to solve tasks where fine detail matters.

MERA: A Hierarchical Feature Tree

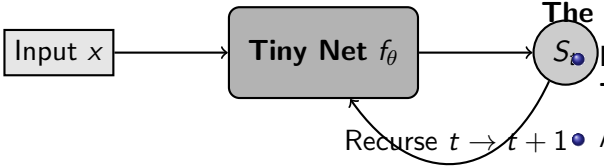
The Data Structure:

- MERA acts as a **Tree of Feature Extractors** built over the input grid.
- **Leaves (Level 0):** Raw pixels.
- **Branches (Levels 1-N):** Successive coarse-graining.
- **Root (Top):** Abstract global concepts.

Logic Flow: Information flows strictly **Upward**. Logic is specialized per layer (local filters $\rightarrow$ global aggregators).



TRM: An Unrolled Reasoning Loop



Meta-Reasoning Interpretation: The network weights θ are **Fixed**. The model applies the *exact same* "thinking rules" at every step.

The Data Structure:

- Instead of a spatial tree, TRM uses a **Temporal Loop**.
- A single "Tiny Network" f_θ processes the state repeatedly.
- **State (S_t):** Contains the current guess (y_t) and "scratchpad" memory (z_t).
- **Deep Supervision:** Explicitly trains the model to improve S_t at every single step, preventing "thought drifting."

The Parallel: Temporal MERA

Core Insight: Iterating a local function over time builds a virtual hierarchy.

Why Time = Scale?

1. **Step 1:** Network sees neighbors (Local repairs).
2. **Step 2:** Network sees neighbors' outputs (Mid-range logic).
3. **Step N:** Information propagates across grid (Global consistency).

Mapping TRM to MERA

TRM (Loop)	MERA (Tree)
Iteration Step (t) *Duration of thinking*	**Tree Depth (L)** *Level of abstraction*
Shared Weights (f_θ) *"Jack-of-all-Trades" Logic*	**Scale Invariance** *Fractal / Self-Similar*
State Evolution (S_t) *Refining the answer*	**Upward Flow** *Coarse-graining data*

Limitation & Fix: Context-Aware Reasoning

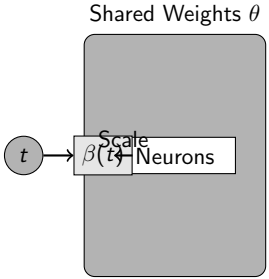
The Problem: Pure TRM uses the *same* logic for Step 1 (Pixel Denoising) and Step 20 (Global Counting). It forces one "brain" to do distinct jobs simultaneously.
The Fix: Modulated TRM (Scale-Dependent Beta Function).

New Mechanism:

$$S_{t+1} = f_\theta(S_t, x, \beta(t))$$

- θ **(Weights):** The **Capability** (Add, Move, Check). Stays fixed.
- $\beta(t)$ **(Modulation):** The **Attention**. Directs the capability based on the phase of reasoning.

"Step 1: Focus on Edges. Step 20: Focus on Symmetry."



Result: A "Smart" Loop that adapts logic to the problem stage.

Part II

Recursive Tensor Networks for ARC-AGI

Proposal for a Tractable & Interpretable Architecture

The Extension: Lightweight Modulation

The Challenge: Standard Recursion forces the same logic for Pixels (Scale 0) and Objects (Scale 5). **The Bloated Solution:** Use separate layers for every scale (Standard CNN/Transformer). *This violates the "Small Model" goal.*
Our Tractable Solution: Hypernetwork Modulation We use a *tiny* side-network (Hypernetwork) to tweak the main weights based on scale s.

$$\theta(s) = \theta_{base} + \Delta(s)$$

$$\Delta(s) = \text{MLP}_{small}(s)$$

Efficiency Win:

- Instead of storing T distinct layers ($T \times |\theta|$ params), we store 1 base layer + 1 tiny MLP ($1 \times |\theta| + \epsilon$).
- **Result:** We get the expressivity of a deep net with the parameter count of a shallow one.

Design Goal: Interpretable by Construction

Most Deep Learning models are "Black Boxes" (Dense Vectors). We design this model to be a "Glass Box" (Geometric Lattice).

Why this is Interpretable:

1. **Spatial Latents:** The hidden state h_t is always a 2D Grid. We can visualize the "thought process" at any step t simply by plotting the tensor heatmaps.
2. **Explicit Hierarchy:** We know exactly which layer handles texture (UV) and which handles logic (IR).
3. **Complexity Metric:** The "Bond Dimension" χ gives us a rigorous number for how "hard" the model finds a specific task.

The Architecture: A Minimalist Recursive Loop

Concept: A single "Reasoning Core" applied iteratively to the grid.

The Flow:

- **Input:** 30×30 PEPS Grid (Categorical Embedding).
- **Loop ($t = 1 \dots T$):**
  - **Modulate:** Generate θ_t using the tiny Hypernetwork.
  - **Apply Core:** Run the MERA Unit (Disentangle + Compress).
- **Output:** The final invariant tensor (The Rule).

Tractability Constraints:

- **Small Tensors:** All contractions are local (4×4 matrices).
- **Low Rank:** We strictly cap the Bond Dimension $\chi \leq 32$.

The Core Unit: Small Matrices, Big Impact

The "Core" consists of two specific linear algebra operations.

1. **The Disentangler (U):**
   - *Size:* A 4×4 Unitary Matrix.
   - *Role:* Rotates the basis to separate "Pattern" from "Noise".
   - *Visual:* Transforms a "Checkerboard" block into "Flat Color + Texture Channel".
2. **The Isometry (W):**
   - *Size:* A 4×2 Projection Matrix.
   - *Role:* Keeps the "Pattern" channel, discards the "Noise" channel.

Total Parameters per Core: ≈ 100 floats. (Extremely small).

Interpretability: The "Holographic" Bottleneck

How do we ensure the model learns *rules* and doesn't just memorize pixels?

The Information Bottleneck: By keeping the Bond Dimension χ low, we force the model to compress data.

- **Scenario A (Memorization):** To pass raw pixel data through the network, you need a massive pipe ($\chi = 1000$). *Our architecture forbids this.*
- **Scenario B (Reasoning):** To pass the *rule* "Shift Left", you only need a tiny pipe ($\chi = 2$).

Result: The architecture physically prevents memorization of complex noise.

Solving the Bond Dimension Dilemma

The Conflict:

- **Too Big ($\chi = 32$):** Dense contraction is $O(\chi^6)$ to $O(\chi^{10})$. Intractable.
- **Too Small ($\chi = 4$):** Network lacks capacity to store complex object relations.

The Solution: Block-Sparse Symmetries ($U(1)$ or $\mathbb{Z}_{10}$) We do not use dense tensors. We enforce conservation laws (e.g., "Mass/Color Conservation") to block-diagonalize the tensors.

$$T_{dense} \approx \begin{pmatrix} \blacksquare & \cdot & \cdot \\ \cdot & \blacksquare & \cdot \\ \cdot & \cdot & \blacksquare \end{pmatrix}$$

Without Symmetry ($\chi = 32$)

- **Params:** $32^4 \approx 1,000,000$
- **Compute:** Extremely Slow.
- **Risk:** Learning valid rules is hard amidst noise.

With Symmetry ($\chi_{eff} = 32$)

- **Structure:** Eight blocks of size $\chi = 4$.
- **Params:** $8 \times 4^4 \approx 2,000$
- **Benefit:** High logical capacity, low compute cost.

Optimization: Finding the Simplest Program

We train using a loss function that penalizes complexity (Occam's Razor).

$$\mathcal{L} = \underbrace{-\log P(Y|X)}_{\text{Accuracy}} + \lambda \sum_t \text{Entropy}(\text{Bond}_t)$$

- **Accuracy:** Did we solve the puzzle?
- **Entropy Penalty:** Did we use the fewest possible bits to do it?

Outcome: The model is incentivized to find the "Simplest Program" (lowest χ) that explains the data, which correlates with strong generalization.

Inference: Search instead of Depth

Standard large models rely on depth (billions of ops) to solve problems. We rely on **Search** (Test-Time Training).

The Process:

1. **Load:** Load the tiny pre-trained Core (θ_{base}).
2. **Adapt:** For the specific test task, run a few steps of optimization on the Hypernetwork input.
3. **Converge:** "Tune" the model to the specific physics of that grid.

Tractability: Because the model is tiny, we can run hundreds of gradient steps *during inference* in seconds.

Execution Roadmap: Jan - April 2026

Phase 1: The Substrate (Jan 1-31)

- **Week 1-2:** Implement 'PEPS' class in `TensorNetwork` library.
- **Week 3:** Implement block-sparse contraction to handle empty space.
- **Week 4:** Validation Test: Confirm Area Law scaling on simple geometric shapes.

Phase 2: The Engine (Feb 1-28)

- **Week 1-2:** Build the MERA Unit (Unitary Disentanglers + Isometries).
- **Week 3:** Implement the Modulation Hypernetwork $\mathcal{M}(s)$.
- **Week 4:** Unit Test: Train the core to compress/reconstruct a fixed square.

Phase 3: The Search (Mar 1-31)

- **Week 1:** Implement Holographic Loss (Likelihood + Bond Entropy).
- **Week 2-3:** Build the Test-Time Training (TTT) loop with Langevin Dynamics.
- **Week 4:** Integration: End-to-end run on ARC "Easy" subset.

Phase 4: Completion (Apr 1-30)

- **Week 1-2:** Full Benchmark on ARC evaluation set.
- **Week 3:** Generate tensor visualization plots (Interpretability).
- **Week 4:** Final Thesis/Report

Expected Evaluation Metrics

1. **Parameter Count:** We aim for $< 10M$ parameters (vs. LLMs with billions).
2. **Interpretability Score:** Can a human look at the intermediate tensor maps and describe the logic? (e.g., "Layer 2 detected the blue corners").

Conclusion: Small, Smart, Structured

Summary: We propose a **Geometric Recursive Network** that solves ARC via structure, not scale.

Key Value Props:

- **Tractable:** Runs on consumer hardware (Small θ, Sparse Ops).
- **Interpretable:** Visualizable states and geometric logic.
- **Generalizable:** Constrained search space prevents memorization.